

Learning to Read x86 Assembly Like a Reverse Engineer

A Semantic Recovery Workbook for OSED Students

OSED-Toolkit

2026-07-21

Contents

Preface and Study Method	5
1 Reading Assembly as Semantic Evidence	8
1.1 Learning objectives	8
1.2 Concept discussion	8
1.3 Common compiler patterns	8
1.4 Fully annotated example	9
1.5 Reverse engineering thought process	10
1.6 Common mistakes	10
1.7 Exercises	10
1.8 Challenge problems	11
1.9 Solutions with reasoning	11
2 Frames, Arguments, Locals, CALL, and RET	13
2.1 Learning objectives	13
2.2 Concept discussion	13
2.3 Common compiler patterns	14
2.4 Fully annotated example	14
2.5 Reverse engineering thought process	15
2.6 Common mistakes	15
2.7 Exercises	15
2.8 Challenge problems	16
2.9 Solutions with reasoning	16
3 LEA, MOVXX, MOVZX, TEST, CMP, and SETcc	18
3.1 Learning objectives	18
3.2 Concept discussion	18
3.3 Common compiler patterns	18
3.4 Fully annotated example	19
3.5 Reverse engineering thought process	19
3.6 Common mistakes	19
3.7 Exercises	20
3.8 Challenge problems	20
3.9 Solutions with reasoning	20
4 Branches, Conditions, and Nested Decisions	22
4.1 Learning objectives	22

4.2	Concept discussion	22
4.3	Common compiler patterns	22
4.4	Fully annotated example	23
4.5	Reverse engineering thought process	23
4.6	Common mistakes	24
4.7	Exercises	24
4.8	Challenge problems	24
4.9	Solutions with reasoning	25
5	Loops and Repeated Work	26
5.1	Learning objectives	26
5.2	Concept discussion	26
5.3	Common compiler patterns	26
5.4	Fully annotated example	27
5.5	Reverse engineering thought process	27
5.6	Common mistakes	28
5.7	Exercises	28
5.8	Challenge problems	28
5.9	Solutions with reasoning	29
6	Switches, Jump Tables, Tail Calls, and Thunks	30
6.1	Learning objectives	30
6.2	Concept discussion	30
6.3	Common compiler patterns	30
6.4	Fully annotated example	31
6.5	Reverse engineering thought process	31
6.6	Common mistakes	32
6.7	Exercises	32
6.8	Challenge problems	32
6.9	Solutions with reasoning	32
7	Data Flow: Pointers, Aliases, Stack, Heap, and Globals	34
7.1	Learning objectives	34
7.2	Concept discussion	34
7.3	Common compiler patterns	34
7.4	Fully annotated example	35
7.5	Reverse engineering thought process	36
7.6	Common mistakes	36
7.7	Exercises	36
7.8	Challenge problems	36
7.9	Solutions with reasoning	37
8	Structures, Arrays, Strings, and Buffer Semantics	38
8.1	Learning objectives	38
8.2	Concept discussion	38
8.3	Common compiler patterns	38
8.4	Fully annotated example	39
8.5	Reverse engineering thought process	40

8.6	Common mistakes	40
8.7	Exercises	40
8.8	Challenge problems	40
8.9	Solutions with reasoning	41
9	MSVC Idioms and Runtime Artifacts	42
9.1	Learning objectives	42
9.2	Concept discussion	42
9.3	Common compiler patterns	42
9.4	Fully annotated example	43
9.5	Reverse engineering thought process	44
9.6	Common mistakes	44
9.7	Exercises	44
9.8	Challenge problems	45
9.9	Solutions with reasoning	45
10	Methodology: From Unknown Function to Working Model	46
10.1	Learning objectives	46
10.2	Concept discussion	46
10.3	Common compiler patterns	46
10.4	Fully annotated example	47
10.5	Reverse engineering thought process	48
10.6	Common mistakes	48
10.7	Exercises	48
10.8	Challenge problems	49
10.9	Solutions with reasoning	49
11	Exploit-Oriented Reading	50
11.1	Learning objectives	50
11.2	Concept discussion	50
11.3	Common compiler patterns	50
11.4	Fully annotated example	51
11.5	Reverse engineering thought process	51
11.6	Common mistakes	52
11.7	Exercises	52
11.8	Challenge problems	52
11.9	Solutions with reasoning	53
12	Case Studies from Vulnerable Windows Service Patterns	54
12.1	Learning objectives	54
12.2	Concept discussion	54
12.3	Common compiler patterns	54
12.4	Fully annotated example	55
12.5	Reverse engineering thought process	56
12.6	Common mistakes	56
12.7	Exercises	56
12.8	Challenge problems	57
12.9	Solutions with reasoning	58

12.10	Final capstone	58
12.11	Appendix A: Exercise answering template	60
12.12	Appendix B: Fast semantic checklist	60
12.13	Appendix C: WinDbg verification prompts	61
13	Printable Worksheets and Checklists	62
14	Shellcode Idioms and Position-Independent Patterns	73
14.1	Learning objectives	73
14.2	Concept discussion	73
14.3	Common shellcode patterns	74
14.4	Fully annotated example	74
14.5	Reverse engineering thought process	76
14.6	Common mistakes	76
14.7	Exercises	76
14.8	Challenge problems	77
14.9	Solutions with reasoning	77
15	ROP Gadgets and Code-Reuse Patterns	79
15.1	Learning objectives	79
15.2	Concept discussion	79
15.3	Common gadget patterns	79
15.4	Fully annotated example	80
15.5	Reverse engineering thought process	82
15.6	Common mistakes	83
15.7	Exercises	83
15.8	Challenge problems	84
15.9	Solutions with reasoning	84

Preface and Study Method

This workbook is for the stage where assembly syntax is no longer the main problem. You can read registers, stack references, addressing modes, calls, and conditional jumps. The hard part is deciding what those facts mean.

An OSED-level reverse engineer is not trying to produce pretty C. The job is to recover enough of the programmer's model to answer security questions:

- What did the programmer think this value represented?
- What did the compiler preserve, erase, or transform?
- Where does attacker-controlled data enter?
- Which checks make a value trusted?
- Which checks only protect a different object?
- What memory is read or written because of that value?
- What control-flow edge becomes reachable?

The difference matters. A decompiler may tell you a function calls `memcpy`. That is not the answer. The answer is whether the destination, source, and count form a valid relationship under all reachable inputs.

0.0.1 How to work each chapter

Read each chapter in four passes.

First pass: skim the concept discussion and pattern list. Do not memorize the patterns as signatures. Use them as expectations to test against evidence.

Second pass: read the annotated example with a notebook open. For every compare, write the question being asked. For every memory operand, write whether it is a read, write, address calculation, stack slot, structure field, array element, or indirect control-flow source.

Third pass: do the exercise without looking at the solution. Do not write pseudocode first. Fill out a fact ledger:

```
Inputs:
Outputs:
Important reads:
Important writes:
Branch questions:
Validated ranges:
Compiler artifacts:
```

Programmer assumptions:
Exploit relevance:
Unresolved facts:

Fourth pass: compare your answer to the solution. If your pseudocode differs but your facts match, that is usually acceptable. If your facts differ, fix the reasoning before moving on.

0.0.2 Evidence levels

Not every conclusion has the same strength.

Strong evidence:

- A memory write destination and count.
- A call target and pushed arguments.
- A conditional jump tied to a specific `cmp` or `test`.
- A dereference proving a value is used as a pointer.
- An immediate size used as a copy bound or allocation size.
- A field offset used consistently across functions.

Medium evidence:

- A return convention such as `0` for success and `-1` for failure.
- A repeated offset suggesting a structure field.
- A string constant used near dispatch code.
- An import name such as `recv`, `send`, `malloc`, `free`, or `VirtualProtect`.

Weak evidence:

- A decompiler variable name.
- A guessed source-level type.
- A familiar-looking pattern without destination/count proof.
- A function name from an old symbol file or analyst comment.

The workbook solutions deliberately separate facts from hypotheses. Keep that habit. It prevents the most common reverse engineering failure: explaining code you have not actually proven.

0.0.3 What to ignore first

Experienced reversers ignore aggressively, but they do it after classification.

Usually ignore on the first pass:

- standard frame setup and teardown
- callee-saved register preservation
- stack-cookie setup (`mov eax, __security_cookie; xor eax, ebp; mov [ebp-4], eax` and the matching check before `retn`) until exploit impact is being considered (chapter 9 covers cookie patterns in detail)
- logging calls unless attacker data becomes a format string
- response formatting unless it leaks useful information
- import thunks once the API boundary is identified

Do not ignore:

- argument setup before calls
- copies into stack or heap buffers
- compare signedness
- length/capacity relationships
- indirect calls or jumps
- stores through attacker-influenced pointers
- exception registration in stack-corruption contexts

0.0.4 How a reverse engineer thinks differently than a compiler

The compiler already solved a different problem: preserve program behavior while obeying the ABI and optimization rules. It does not preserve source intent for your convenience. It may remove variables, merge branches, reuse stack slots, inline routines, tail-call handlers, or replace multiplication with `lea`.

The reverse engineer solves a semantic problem: identify the model that makes the machine behavior coherent.

The exploit developer solves a narrower adversarial problem: identify where attacker-controlled values violate the programmer's assumed invariants and turn that violation into control, disclosure, or corruption.

Those mindsets overlap, but they are not identical. A reverse engineer may stop after proving a parser accepts a record. An exploit developer keeps going until they know whether a field controls a write size, write address, function pointer, saved return address, SEH record, heap metadata, or ROP-relevant state.

0.0.5 Print workflow

For each exercise page, cover the solution section. Write your answer in this order:

1. One-sentence purpose hypothesis.
2. Facts that support it.
3. Facts that weaken it.
4. Values that are attacker-controlled or unknown.
5. Pseudocode only after the above is complete.

If you cannot explain why the compiler emitted an instruction, mark it as unresolved. Do not invent a source statement just to make the disassembly feel complete.

Chapter 1

Reading Assembly as Semantic Evidence

1.1 Learning objectives

By the end of this chapter you should be able to:

- Separate compiler scaffolding from programmer intent.
- Read an instruction sequence as information flow instead of syntax.
- Maintain a fact ledger without prematurely inventing C source.
- Explain what question the CPU is answering at each decision point.

1.2 Concept discussion

Reverse engineering is not decompilation by hand. A compiler has already erased source-level names, comments, some types, and many boundaries between statements. What remains is executable evidence. Your job is to recover the simplest model that explains that evidence.

The CPU does not know about “valid packets”, “users”, or “commands”. It answers smaller questions:

- Is this value zero?
- Is this unsigned length larger than a limit?
- What address is $\text{base} + \text{index} * \text{scale} + \text{offset}$?
- Which function address should be called?
- How many bytes should be copied?

The programmer wrote logic in terms of domain concepts. The compiler lowered that logic into register movement, memory access, condition flags, and control flow. The reverse engineer climbs back up from CPU questions to programmer assumptions.

1.3 Common compiler patterns

- Save/restore registers: compiler obligation, usually not domain logic.

- `xor eax, eax`: often materializes zero, frequently a success return or boolean false.
- `cmp reg, imm` followed by conditional jump: a branch question.
- `test reg, reg`: zero/null/empty check.
- `lea reg, [mem]`: address or arithmetic, not a memory read.
- `push arg`; `call target`; `add esp, n`: argument setup and caller cleanup.

1.4 Fully annotated example

Assembly first:

```
sub_401000:
    push    ebp
    mov     ebp, esp
    mov     eax, [ebp+8]
    test    eax, eax
    jz      short fail
    cmp     dword ptr [eax], 4D434D44h
    jnz     short fail
    xor     eax, eax
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFFh
    pop     ebp
    retn
```

Annotated reading:

```
push ebp / mov ebp, esp
; Standard frame. This tells us where arguments live, not what the program
; means.

mov eax, [ebp+8]
; Load the first stack argument.

test eax, eax
; Ask: is the argument zero? Since the next use dereferences EAX, this is a
; null
; pointer check.

jz fail
; If the pointer is null, return failure.

cmp dword ptr [eax], 4D434D44h
; Read four bytes from the pointed-to object and compare them to a constant.
; The argument is a pointer to a structure or buffer whose first dword is a
; tag.
```

```
jnz fail
; If the tag does not match, return failure.

xor eax, eax
; Return 0. In Windows C code, 0 commonly means success for internal
  ↪ helpers.
```

The CPU is answering: “Did I receive a non-null pointer whose first four bytes match the expected tag?”

The programmer likely wrote a validation helper. The compiler assumes `[ebp+8]` is the only argument used here and that reading `[eax]` is safe after the null check. The programmer assumes the caller passed a pointer to at least four readable bytes.

1.5 Reverse engineering thought process

Start with facts:

- One argument is read from `[ebp+8]`.
- That value is checked for zero.
- If nonzero, four bytes at that address are read.
- Two paths return: `0` and `-1`.

Then hypotheses:

- Argument is a pointer.
- The first dword is a magic, signature, opcode, or type tag.
- The function is a validator.

Do not name it `check_header` yet unless cross-references prove the domain.

1.6 Common mistakes

- Treating `test eax, eax` as arithmetic. It is usually a question about zero.
- Calling the constant a string without checking endianness and context.
- Ignoring the failure return convention.
- Assuming the pointer is attacker-controlled before walking callers.

1.7 Exercises

Assembly only:

```
sub_401050:
    mov     eax, [esp+4]
    test    eax, eax
    jz      short bad
    cmp     byte ptr [eax+2], 1
    jnz     short bad
```

```
    mov     eax, 1
    retn
bad:
    xor     eax, eax
    retn
```

Answer before reading the solution:

- What does this code accomplish?
- Why does it exist?
- What data is important?
- What can be ignored?

1.8 Challenge problems

```
sub_401080:
    push    ebp
    mov     ebp, esp
    mov     ecx, [ebp+8]
    xor     eax, eax
    test    ecx, ecx
    jz      short done
    cmp     word ptr [ecx], 5A4Dh
    setz    al
done:
    pop     ebp
    retn
```

Explain why this is not merely a “compare function”. Identify the programmer’s likely assumption about the pointed-to data.

1.9 Solutions with reasoning

Exercise solution:

The function reads one argument from `[esp+4]`, checks it for null, reads one byte at offset +2, and returns 1 only if that byte equals 1. It likely tests a field inside a small structure or record.

Plausible pseudocode, introduced only after the reasoning:

```
int is_enabled(const unsigned char *p) {
    if (!p) return 0;
    return p[2] == 1;
}
```

The important data is the pointer and byte offset +2. The absence of an EBP frame is not important by itself; this is still a normal function.

Challenge solution:

The code asks whether a non-null pointer begins with the word value 0x5A4D, which represents ASCII bytes M (0x4D) and Z (0x5A) as loaded on a little-endian machine. `setz al` materializes the comparison result as a boolean. The programmer assumes the pointer references at least two readable bytes. An exploit developer asks whether that pointer can point near an invalid page, because no structured exception handling appears in the function.

Chapter 2

Frames, Arguments, Locals, CALL, and RET

2.1 Learning objectives

- Recover stack-frame shape and argument count from x86 code.
- Distinguish locals, saved registers, outgoing arguments, and return values.
- Infer likely calling convention from cleanup behavior.
- Recognize when frame code is compiler scaffolding.

2.2 Concept discussion

A function frame is the compiler's workspace. It exists because the programmer needed locals, calls, preserved registers, or stable stack references. It does not necessarily mean the programmer consciously used the stack.

The frame answers:

- Where can the compiler keep local state?
- Which registers must survive this function?
- Who removes arguments from the stack after a call?
- Where does the return value appear?

In 32-bit MSVC, a debug or non-optimized function often uses EBP as a stable frame pointer:

```
push ebp
mov  ebp, esp
sub  esp, local_size
...
mov  esp, ebp
pop  ebp
retn
```

Arguments appear above EBP: [ebp+8], [ebp+0Ch], [ebp+10h]. Locals appear below EBP: [ebp-4], [ebp-20h].

2.3 Common compiler patterns

- `push esi / pop esi`: callee-saved register preservation.
- `sub esp, 80h`: local stack allocation.
- `lea eax, [ebp-40h]`: address of a local buffer.
- `push [ebp+0Ch]; push [ebp+8]; call target; add esp, 8`: cdecl-style caller cleanup.
- `retn 8`: callee cleanup, common in stdcall.
- `mov eax, imm` before epilogue: return value.

2.4 Fully annotated example

```
sub_401100:
    push    ebp
    mov     ebp, esp
    sub     esp, 44h
    push    esi
    mov     esi, [ebp+8]
    lea     eax, [ebp-40h]
    push    40h
    push    esi
    push    eax
    call    _memcpy
    add     esp, 0Ch
    xor     eax, eax
    pop     esi
    mov     esp, ebp
    pop     ebp
    retn
```

Annotated:

```
sub esp, 44h
; Reserve 0x44 bytes of local storage. The interesting object later starts
; ↪ at
; ebp-0x40, so four bytes may be padding, a temp, or alignment.

push esi
; Preserve ESI because this function uses it and must restore it.

mov esi, [ebp+8]
; First argument copied into ESI. Later passed as source, so it is probably
; ↪ a
; source pointer.

lea eax, [ebp-40h]
; Compute address of local buffer. LEA does not read from the buffer.
```

```

push 40h / push esi / push eax / call _memcpy
; Arguments are pushed right-to-left: memcpy(dst=&local, src=arg0,
  ↳ len=0x40).

add esp, 0Ch
; Caller cleaned three 4-byte arguments.

```

The CPU is answering: “Copy exactly 64 bytes from the caller-supplied pointer into a local stack buffer, then report success.”

2.5 Reverse engineering thought process

First draw the frame:

```

[ebp+8]   arg0: source pointer
[ebp+4]   return address
[ebp]     saved EBP
[ebp-4]   unknown/padding/temp
[ebp-40h] local buffer begins

```

Then separate scaffolding from semantics:

- Frame setup/teardown: scaffolding.
- push esi/pop esi: scaffolding.
- lea [ebp-40h], push 40h, _memcpy: semantics.

2.6 Common mistakes

- Treating every stack slot as a distinct programmer variable.
- Assuming `sub esp, 44h` means a 68-byte buffer. The used buffer may be smaller.
- Forgetting that pushes before a call are usually outgoing arguments.
- Ignoring whether `ret` has an immediate operand.

2.7 Exercises

Assembly only:

```

sub_401160:
  push    ebp
  mov     ebp, esp
  sub     esp, 20h
  lea     eax, [ebp-20h]
  push    eax
  push    [ebp+8]
  call    sub_402000

```



```
add     esp, 8
mov     esp, ebp
pop     ebp
retn
```

Questions:

- How many explicit stack arguments does this function use?
- Which stack object is passed by address?
- Who cleans up the call to sub_402000?
- What can you infer about the called function's parameters?

2.8 Challenge problems

```
sub_4011A0:
push    ebp
mov     ebp, esp
push    edi
mov     edi, [ebp+8]
push    [ebp+0Ch]
call    dword ptr [edi+10h]
pop     edi
pop     ebp
retn    8
```

Explain the calling convention of sub_4011A0 and the nature of the call target.

2.9 Solutions with reasoning

Exercise solution:

The function uses one explicit incoming argument, [ebp+8]. It creates a 32-byte local object and passes its address to sub_402000, along with the incoming argument. `add esp,8` means this function cleans the two outgoing arguments after the call, so the target is being called as cdecl or through a cdecl-compatible prototype.

Plausible pseudocode:

```
void wrapper(void *arg) {
    unsigned char local[32];
    sub_402000(arg, local);
}
```

Challenge solution:

`retn 8` means sub_4011A0 cleans two caller-supplied arguments, consistent with stdcall or thiscall-like external usage. The call target is loaded from [edi+10h], where EDI came from the first argument. That is an indirect call through a structure field or vtable-like slot. Because this

function pushes one outgoing argument before the indirect call and does not execute `add esp, 4` afterward, the indirect callee must clean that argument, for example by returning with `ret 4`. If it were `cdecl, pop edi` would consume the stale argument instead of the saved EDI and the frame would unwind incorrectly. The programmer likely wrote a callback or method dispatch. An exploit developer asks whether `[edi+10h]` can be controlled.

Chapter 3

LEA, MOVSX, MOVZX, TEST, CMP, and SETcc

3.1 Learning objectives

- Explain why `lea` often means arithmetic rather than pointer dereference.
- Use `movsx` and `movzx` to infer signedness and source width.
- Read `test`, `cmp`, and `setcc` as boolean and range questions.
- Avoid turning flag-setting instructions into fake source statements.

3.2 Concept discussion

These instructions are semantic clues. They reveal the compiler's type and range assumptions:

- `movzx eax, byte ptr [ecx]`: source is an unsigned byte promoted to int.
- `movsx eax, word ptr [ecx]`: source is a signed short promoted to int.
- `test eax, eax`: check zero, null, or false.
- `cmp eax, 80h + ja`: unsigned greater-than.
- `cmp eax, 80h + jg`: signed greater-than.
- `setnz al`: materialize a boolean result into a byte.

The compiler emits these because C does not compare abstract “values”; it compares values of a width and signedness after promotion.

3.3 Common compiler patterns

- `lea eax, [ecx+ecx*4]`: multiply by 5.
- `lea eax, [edx+eax*4+8]`: array element address or structure-member address.
- `movzx eax, al`: clear high bits after byte arithmetic.
- `test reg, reg; jz`: null/zero check.
- `cmp reg, imm; setcc al; movzx eax, al`: return boolean as int.

3.4 Fully annotated example

```
sub_401220:
    mov     ecx, [esp+4]
    movsx   eax, word ptr [ecx+4]
    lea     edx, [eax+eax*2]
    cmp     edx, 30h
    setle   al
    movzx   eax, al
    retn
```

Annotated:

```
mov  ecx, [esp+4]
; Argument, likely pointer because it is dereferenced next.

movsx eax, word ptr [ecx+4]
; Read a 16-bit signed field at offset +4 and promote it to 32 bits.

lea  edx, [eax+eax*2]
; Compute value * 3. No memory read.

cmp  edx, 30h
setle al
; Signed comparison: is field*3 <= 48?

movzx eax, al
; Return boolean as 0 or 1.
```

The CPU is answering: “Does the signed 16-bit field at offset 4, multiplied by three, fit under a signed threshold?”

The programmer likely wrote a small validation predicate. The compiler assumes signed behavior matters because it chose `movsx` and signed `setle`.

3.5 Reverse engineering thought process

When you see `lea`, ask: “Is the result later dereferenced?” If no, it may just be arithmetic. When you see `movsx/movzx`, record source width and signedness. When you see `setcc`, identify which compare produced the flags and whether the condition is signed or unsigned.

3.6 Common mistakes

- Saying `lea` always means “load pointer.”
- Losing signedness by rewriting everything as `int`.
- Treating `setcc` as a branch; it creates data, not control flow.
- Ignoring high-register cleanup before boolean returns.

3.7 Exercises

```
sub_401250:
    mov     eax, [esp+4]
    movzx   ecx, byte ptr [eax+8]
    lea     edx, [ecx+ecx*4]
    cmp     edx, 64h
    seta    al
    movzx   eax, al
    retn
```

Questions:

- What is the width and signedness of the field?
- What arithmetic is being performed?
- What boolean is returned?

3.8 Challenge problems

```
sub_401280:
    mov     ecx, [esp+4]
    movsx   eax, byte ptr [ecx+1]
    test    eax, eax
    jns     short ok
    neg     eax
ok:
    cmp     eax, 7Fh
    setbe   al
    movzx   eax, al
    retn
```

Recover the programmer-level question without writing a line-by-line translation.

3.9 Solutions with reasoning

Exercise solution:

The field at offset +8 is an unsigned byte. It is multiplied by 5 using LEA. The function returns whether $\text{field} * 5 > 100$ using an unsigned comparison (seta).

Plausible pseudocode:

```
int too_large(const unsigned char *p) {
    return p[8] * 5u > 100u;
}
```

Challenge solution:

The function reads a signed byte at offset +1, computes its absolute value, and returns whether that absolute value is ≤ 127 . The interesting semantic clue is not `neg`; it is the pair `movsx` and `jns`, which shows the original byte was signed. The programmer likely intended an absolute-value range check. An exploit developer asks about the edge case -128 , because negating the most negative signed byte has representation pitfalls even though this version promoted to 32-bit first.

Chapter 4

Branches, Conditions, and Nested Decisions

4.1 Learning objectives

- Recover if, if/else, guard clauses, and nested branches.
- Determine what condition controls each path.
- Preserve signedness while reconstructing decisions.
- Identify validation gates and trust boundaries.

4.2 Concept discussion

Branches are questions. The jump mnemonic tells you how the flags are interpreted. `ja` and `jb` are unsigned. `jg` and `jl` are signed. `jz` tests equality/zero. `jnz` tests inequality/nonzero.

A compiler often emits guard clauses as early jumps to failure. This means the source may have looked simple even when the assembly appears branch-heavy:

```
if (!p) return -1;
if (len > max) return -1;
if (!authorized) return -1;
...
return 0;
```

The reverse engineer should read these as successive requirements, not as unrelated fragments.

4.3 Common compiler patterns

- `test p,p; jz fail:` null guard.
- `cmp len,max; ja fail:` unsigned length guard.
- `cmp status,0; jl fail:` signed error check.
- `cmp type,n; jne next_case:` decision chain.
- Common failure block with `mov eax,-1:` validation helper.

4.4 Fully annotated example

```

sub_401300:
    push    ebp
    mov     ebp, esp
    mov     eax, [ebp+8]
    test    eax, eax
    jz      fail
    mov     ecx, [ebp+0Ch]
    cmp     ecx, 100h
    ja      fail
    cmp     byte ptr [eax], 2
    jnz     fail
    xor     eax, eax
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFFh
    pop     ebp
    retn

```

Annotated:

```

test eax, eax / jz fail
; Requirement 1: pointer argument must be non-null.

cmp ecx, 100h / ja fail
; Requirement 2: unsigned length must be <= 256.

cmp byte ptr [eax], 2 / jnz fail
; Requirement 3: first byte of pointed-to buffer must equal type 2.

```

The CPU is answering three guard questions. The programmer likely wrote a validator for a record. The compiler assumes `len` is unsigned or size-like because it used `ja`.

4.5 Reverse engineering thought process

Rewrite branches into path requirements:

```

success path requires:
    p != NULL
    len <= 0x100 unsigned
    p[0] == 2
failure path:
    any requirement fails

```

This is more useful than immediately writing C because it exposes which values become trusted after each check.

4.6 Common mistakes

- Inverting a condition incorrectly when following taken vs fall-through edges.
- Losing unsigned/signed distinction.
- Treating each guard as independent after a prior guard has constrained a value.
- Missing that a dereference after a guard depends on the guard.

4.7 Exercises

```
sub_401340:
    mov     eax, [esp+4]
    test    eax, eax
    jz      deny
    cmp     dword ptr [eax+0Ch], 0
    jle     deny
    cmp     dword ptr [eax+10h], 400h
    ja      deny
    mov     eax, 1
    retn
deny:
    xor     eax, eax
    retn
```

Questions:

- Which comparisons are signed and unsigned?
- What requirements must hold for success?
- Which fields are probably length/count-like?

4.8 Challenge problems

```
sub_401380:
    mov     eax, [esp+4]
    cmp     eax, 3
    ja      default
    cmp     eax, 1
    jz      one
    cmp     eax, 2
    jz      two
    xor     eax, eax
    retn
one:
    mov     eax, 10h
    retn
two:
    mov     eax, 20h
```

```
    retn
default:
    mov     eax, 0FFFFFFFFh
    retn
```

Explain why the first `cmp eax,3; ja default` matters even though only cases 1 and 2 have special code.

4.9 Solutions with reasoning

Exercise solution:

Success requires a non-null pointer, a signed positive field at `+0x0C`, and an unsigned field at `+0x10` no greater than `0x400`. The second field is probably size-like because of the unsigned comparison.

Plausible pseudocode:

```
int allowed(const struct X *x) {
    if (!x) return 0;
    if (x->signed_count <= 0) return 0;
    if (x->length > 0x400u) return 0;
    return 1;
}
```

Challenge solution:

The first comparison limits the accepted domain to `0..3` unsigned. It prevents large unsigned values from falling into the later chain. Semantically, the programmer is bounding an opcode or enum before handling selected cases. Values `0` and `3` are both unhandled in-range values that fall through to the zero return. An exploit developer asks whether those mappings are intentional default behavior or accidental gaps in the dispatch logic.

Chapter 5

Loops and Repeated Work

5.1 Learning objectives

- Identify loop header, body, update, and exit condition.
- Distinguish counted loops from sentinel loops.
- Recover induction variables and loop invariants.
- Recognize common scan/copy/compare patterns.

5.2 Concept discussion

Loops answer repeated questions. A counted loop asks, “Have I processed N elements?” A sentinel loop asks, “Have I reached the marker?” A traversal asks, “Is there another node/record?”

The compiler may place the condition at the top, bottom, or split across blocks. Optimized loops may not look like source loops. Focus on:

- Which value changes each iteration?
- Which memory address changes?
- Which condition exits?
- What remains true throughout the loop?

5.3 Common compiler patterns

- `xor ecx,ecx; ... inc ecx; cmp ecx,len; jb loop`: counted loop.
- `mov al,[ecx]; test al,al; jz done; inc ecx`: string scan.
- `rep movsb`: fixed-count byte copy.
- `repne scasb`: sentinel scan, often `strlen`-like.
- Pointer bumping: `add esi,4` instead of index arithmetic.

5.4 Fully annotated example

```
sub_401400:
    mov     edx, [esp+4]
    xor     eax, eax
loop_top:
    cmp     eax, [esp+8]
    jae     done
    cmp     byte ptr [edx+eax], 0
    jz      done
    inc     eax
    jmp     short loop_top
done:
    retn
```

Annotated:

```
mov     edx, [esp+4]
; Buffer pointer.

xor     eax, eax
; Index/count starts at zero.

cmp     eax, [esp+8] / jae done
; Unsigned bound check: stop when index >= limit.

cmp     byte ptr [edx+eax], 0 / jz done
; Sentinel check: stop on NUL.

inc     eax
; One byte consumed.
```

The CPU is answering: “How many bytes until either the maximum length is reached or a zero byte appears?”

The programmer wrote bounded string scanning. The compiler preserves two assumptions: the limit is unsigned, and the buffer is readable up to either the terminator or the limit.

5.5 Reverse engineering thought process

Classify the loop:

```
induction variable: eax
base pointer: edx
access pattern: byte [edx+eax]
upper bound: [esp+8]
sentinel: 0
return: number of bytes scanned
```

Then derive intent: bounded length calculation, not generic looping.

5.6 Common mistakes

- Calling every byte loop `strlen`; this one has a maximum bound.
- Missing that `EAX` is both index and return value.
- Treating `[esp+8]` as signed because it “looks like an int.”
- Ignoring the order of checks. Here, the bound is checked before dereference.

5.7 Exercises

```
sub_401450:
    mov     esi, [esp+4]
    mov     ecx, [esp+8]
    xor     eax, eax
again:
    test    ecx, ecx
    jz      done
    add     eax, [esi]
    add     esi, 4
    dec     ecx
    jmp     short again
done:
    retn
```

Questions:

- What is the element size?
- Which value is the induction mechanism?
- What does the function return?

5.8 Challenge problems

```
sub_401490:
    xor     eax, eax
    or      ecx, 0FFFFFFFFh
    repne   scasb
    not     ecx
    dec     ecx
    mov     eax, ecx
    retn
```

Assume `EDI` points to the input string on entry because the caller set it before a tail call. This is common in inlined or tail-called helpers where the compiler leaves `EDI` set from a prior operation. Recover the pattern and explain the hidden assumption.

5.9 Solutions with reasoning

Exercise solution:

The function processes dword elements because it reads `[esi]` and advances by four. ECX is a countdown. EAX accumulates a sum. The function returns the sum of count 32-bit integers from the array pointer.

Plausible pseudocode:

```
unsigned sum(const unsigned *p, unsigned count) {  
    unsigned total = 0;  
    while (count-->0) total += *p++;  
    return total;  
}
```

Challenge solution:

This is the classic `strlen` scan idiom using `repne scasb`: set AL to zero, set ECX to `0xFFFFFFFF` as the maximum scan count, scan for the NUL byte through memory at EDI, then invert/decrement ECX to produce the length. `scasb` advances EDI and decrements ECX; it does not update EAX with the count. The hidden programmer assumption is that the string is NUL-terminated in readable memory. An exploit developer cares because missing terminators can turn a scan into an out-of-bounds read before any write occurs.

Chapter 6

Switches, Jump Tables, Tail Calls, and Thunks

6.1 Learning objectives

- Recognize switch lowering and jump tables.
- Distinguish dispatcher logic from handler logic.
- Identify tail calls and import thunks.
- Treat indirect branches as control-flow data flow.

6.2 Concept discussion

Dispatch code often exists to route work, not to perform the work. A reverse engineer should avoid spending too long inside scaffolding once the dispatch key, range check, and targets are understood.

Switch statements can lower into:

- Compare chains for sparse cases.
- Jump tables for dense cases.
- Lookup tables plus indirect calls.
- Tail calls when the wrapper simply forwards control.

The CPU is answering: “Given this key, which code address should execute next?”

6.3 Common compiler patterns

- `cmp eax, max; ja default; jmp dword ptr [table+eax*4]`: dense switch.
- `jmp _ImportedFunction`: import thunk.
- `call dword ptr [ecx+offset]`: callback, method, or function pointer field.
- `jmp target` at function end: tail call.
- `mov ecx, this; call dword ptr [eax+N]`: virtual function call pattern.

6.4 Fully annotated example

```
sub_401500:
    mov     eax, [esp+8]
    cmp     eax, 2
    ja      default
    jmp     dword ptr ds:off_402000[eax*4]
case0:
    jmp     handle_ping
case1:
    jmp     handle_echo
case2:
    jmp     handle_copy
default:
    mov     eax, 0FFFFFFFFh
    retn
```

Annotated:

```
mov eax, [esp+8]
; Dispatch key, likely opcode or enum.

cmp eax, 2 / ja default
; Unsigned range check: only 0, 1, 2 are valid table indexes.

jmp [table+eax*4]
; Indirect branch through a dword table of code addresses.

jmp handle_*
; Tail jump to handler. The dispatcher does not return through these
↪ blocks.
```

The programmer likely wrote a switch over an opcode. The compiler assumes the range check protects the table access. The reverse engineer should now analyze the handlers, not overfit the dispatcher.

6.5 Reverse engineering thought process

Recover dispatch in this order:

1. Identify the dispatch key.
2. Identify the valid range.
3. Enumerate targets.
4. Determine whether targets are calls or tail jumps.
5. Walk the target that receives attacker-controlled data.

6.6 Common mistakes

- Treating jump-table data as code.
- Missing the unsigned range check that protects the table.
- Thinking a tail call returns to the dispatcher.
- Assuming indirect call means virtual function without checking object shape.

6.7 Exercises

```
sub_401560:
    mov     ecx, [esp+4]
    mov     eax, [ecx]
    push    [esp+8]
    call    dword ptr [eax+0Ch]
    retn
```

Questions:

- What is the likely role of [ecx]?
- What is at [eax+0Ch]?
- Which value controls the indirect call target?

6.8 Challenge problems

```
j__send:
    jmp     dword ptr ds:__imp__send
```

Explain why this function may appear in cross-references and why spending time inside it is usually wasted.

6.9 Solutions with reasoning

Exercise solution:

ECX is a pointer to an object-like structure. [ecx] loads a table pointer, commonly a vtable. [eax+0x0C] is the fourth function slot. The call target is controlled by the table pointer stored at the object base. An exploit developer asks whether ECX or [ecx] can be corrupted before this call. The snippet pushes one outgoing argument and then returns without add esp, 4, so the indirect callee must use a callee-cleaned convention for that argument. In real thiscall code, also check whether ECX is meant to remain the object pointer at the call site.

Plausible pseudocode:

```
obj->vtable->slot3(arg);
```

Challenge solution:

This is an import thunk. It exists so local code can branch to a stable stub that jumps through the Import Address Table entry for `send`. It contributes API boundary information, not domain logic. The important question is who calls `send`, with which socket, buffer pointer, length, and flags.

Chapter 7

Data Flow: Pointers, Aliases, Stack, Heap, and Globals

7.1 Learning objectives

- Track value origin and destination through registers and memory.
- Recognize pointer propagation and aliases.
- Distinguish stack, heap, global, and imported storage.
- Identify when two names may refer to the same memory.

7.2 Concept discussion

Data flow is the spine of reverse engineering. Control flow tells you which path runs. Data flow tells you what that path means.

A pointer copied from EAX to ESI to [ebp-4] is still the same pointer unless an instruction changes it. A buffer passed into memcpy becomes important because the write destination and count define risk. A global read can be configuration, state, a function table, or a security-sensitive flag.

Reverse engineers care less about variable names than about provenance:

- user input
- parsed input
- validated input
- trusted configuration
- heap allocation
- local stack temporary
- global process state
- imported function pointer

7.3 Common compiler patterns

- `mov esi, [ebp+8]`: keep argument pointer in callee-saved register.

- `mov [ebp-4], eax`: spill a temporary or preserve a value across a call.
- `call _malloc; test eax,eax`: heap allocation and null check.
- `mov eax, ds:g_config`: global read.
- `mov [ecx+offset], eax`: structure field write through pointer.

7.4 Fully annotated example

```

sub_401600:
    push    ebp
    mov     ebp, esp
    push    esi
    mov     esi, [ebp+8]
    push    [ebp+0Ch]
    call    _malloc
    add     esp, 4
    test    eax, eax
    jz      fail
    mov     [esi+8], eax
    mov     ecx, [ebp+0Ch]
    mov     [esi+0Ch], ecx
    xor     eax, eax
    pop     esi
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFFh
    pop     esi
    pop     ebp
    retn

```

Annotated:

```

mov esi, [ebp+8]
; First argument is a destination object pointer.

push [ebp+0Ch] / call _malloc
; Allocate size from second argument.

test eax,eax / jz fail
; Allocation failure check.

mov [esi+8], eax
; Store heap pointer into object field +8.

mov ecx, [ebp+0Ch]
mov [esi+0Ch], ecx
; Store size into object field +0x0C.

```

The function initializes two fields: buffer pointer and buffer size. The programmer assumes the object pointer is valid. The compiler assumes the size argument is already suitable for `malloc`; no range check appears here.

7.5 Reverse engineering thought process

Build a provenance table:

Value	Origin	Destination	Meaning
[ebp+8]	caller	ESI, stores at offsets	object pointer
[ebp+0Ch]	caller	malloc, [object+0Ch]	allocation size
EAX after malloc	heap allocator	[object+8]	heap buffer

Then ask exploit questions: Can size be huge? Can object be fake? Is there later copying into [object+8] using [object+0Ch]?

7.6 Common mistakes

- Losing track of aliases after a pointer is copied.
- Assuming stack memory is safe and heap memory is unsafe; storage class is not trust.
- Treating globals as constants without checking writes.
- Ignoring size fields stored next to pointers.

7.7 Exercises

```
sub_401650:
    mov     eax, [esp+4]
    mov     ecx, ds:g_table
    mov     edx, [eax+4]
    mov     [ecx+edx*4], eax
    xor     eax, eax
    ret
```

Questions:

- What data flows into the global table?
- Which field controls the index?
- What trust questions follow?

7.8 Challenge problems

```
sub_401680:
    mov     eax, [esp+4]
```

```
mov     ecx, [eax+8]
push    ecx
call    _free
add     esp, 4
mov     dword ptr [eax+8], 0
retn
```

Find the aliasing bug in the analyst's reasoning, not necessarily in the program.

7.9 Solutions with reasoning

Exercise solution:

The first argument pointer is stored into a global table. The index comes from the dword field at offset +4 inside that object. If the object or field is attacker-controlled, this may become an out-of-bounds global write or pointer registration primitive.

Plausible pseudocode:

```
g_table[obj->id] = obj;
```

Challenge solution:

After `_free`, EAX is caller-saved and cannot be assumed to still hold the original object pointer. The final `mov [eax+8], 0` is suspicious because the compiler would normally preserve the object pointer across a call if it needed it. Either the snippet is hand-written/incorrectly lifted, there was an unshown preservation instruction, or the compiler inlined `_free` in a link-time optimized build and the inlined body happens not to clobber EAX. The last case is real but dangerous to rely on: observed behavior is not the same as an ABI guarantee. The reverse engineering lesson: do not assume caller-saved registers survive calls, even when a particular build appears to preserve them.

Chapter 8

Structures, Arrays, Strings, and Buffer Semantics

8.1 Learning objectives

- Recover structure fields from repeated offsets.
- Recognize arrays from scale factors and pointer increments.
- Distinguish string copies from raw byte copies.
- Identify length, capacity, and terminator relationships.

8.2 Concept discussion

Structures reveal themselves through stable offsets. Arrays reveal themselves through repeated element size. Strings reveal themselves through sentinel checks or explicit NUL writes.

The core semantic question is not “what is the C type?” It is:

- Which bytes belong together?
- Which value says how many bytes are valid?
- Which value says how many bytes fit?
- Which operation writes a terminator?
- Does the check cover all bytes written?

8.3 Common compiler patterns

- `[ecx+4], [ecx+8], [ecx+0Ch]`: structure fields.
- `[base+index*4]`: dword array, pointer array, or table.
- `mov byte ptr [edi+ecx], 0`: NUL terminator at index.
- `cmp ecx, [ebp+10h]; jae fail`: leaves room for a terminator when ECX is later used as `dst[len]`.
- `rep movsb` or `_memcpy`: raw count-based copy.

8.4 Fully annotated example

```

sub_401700:
    push    ebp
    mov     ebp, esp
    push    esi
    push    edi
    mov     esi, [ebp+8]
    mov     ecx, [esi+4]
    cmp     ecx, [ebp+10h]
    jae     fail
    mov     edi, [ebp+0Ch]
    push    ecx
    push    [esi+8]
    push    edi
    call    _memcpy
    add     esp, 0Ch
    mov     byte ptr [edi+ecx], 0
    xor     eax, eax
    pop     edi
    pop     esi
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFh
    pop     edi
    pop     esi
    pop     ebp
    retn

```

Annotated:

```

mov     esi, [ebp+8]
; Record pointer.

mov     ecx, [esi+4]
; Field +4 is a length.

cmp     ecx, [ebp+10h] / jae fail
; Require length < destination capacity. Strictly less leaves room for NUL.

mov     edi, [ebp+0Ch]
; Destination pointer.

push    ecx / push [esi+8] / push edi / call _memcpy
; Copy record->data, length bytes, into destination.

mov     byte ptr [edi+ecx], 0

```



```
; Terminate string after copied bytes.
```

The programmer wrote “copy record text into a caller buffer if it fits.” The compiler exposes a three-field relationship: source record, destination pointer, destination capacity.

8.5 Reverse engineering thought process

Do not call `[esi+4]` “size” generically. Its role is proven by use:

- Compared against capacity.
- Used as memcpy count.
- Used as terminator index.

That makes it a string length field.

8.6 Common mistakes

- Missing the +1 implied by explicit terminator writes.
- Confusing source length with destination capacity.
- Assuming memcpy means unsafe. Safety depends on the count and destination.
- Treating a structure offset as a constant magic value.

8.7 Exercises

```
sub_401760:  
    mov     eax, [esp+4]  
    mov     ecx, [esp+8]  
    mov     edx, [eax+ecx*8+4]  
    retn
```

Questions:

- What is the apparent element size?
- Is this an array of structures or an array of pointers?
- What field is being read?

8.8 Challenge problems

```
sub_401790:  
    mov     ecx, [esp+0Ch]  
    cmp     ecx, [esp+10h]  
    ja      fail  
    push    ecx  
    push    [esp+0Ch]  
    push    [esp+0Ch]
```

```
call    _memcpy
add     esp, 0Ch
xor     eax, eax
retn
fail:
mov     eax, 0FFFFFFFh
retn
```

Do not trust the apparent stack offsets after pushes. Explain how you would verify the real arguments.

8.9 Solutions with reasoning

Exercise solution:

The scale `*8` indicates 8-byte elements. Reading `[base + index*8 + 4]` suggests an array of structures where field `+4` is loaded from element `index`. If it were an array of pointers, you would expect a scale of 4 followed by a second dereference.

Plausible pseudocode:

```
return records[i].field_4;
```

Challenge solution:

Stack offsets shift as arguments are pushed. A static listing can become confusing if you read `[esp+0Ch]` after previous pushes as if ESP had not moved. In IDA, normalize the prototype and inspect stack deltas; in WinDbg, break before the call and dump `dd esp L8` to identify the actual pushed values. The semantic goal is to prove destination, source, and count, not to memorize offsets in a moving stack.

Chapter 9

MSVC Idioms and Runtime Artifacts

9.1 Learning objectives

- Recognize common MSVC-generated and CRT-like idioms.
- Distinguish programmer logic from compiler-inserted protections.
- Identify copies, clears, string routines, stack cookies, stack probing, SEH, function pointers, virtual dispatch, and import thunks.
- Explain why the compiler generated each pattern.

9.2 Concept discussion

Compiler idioms are shortcuts, not conclusions. You should recognize them, then prove the contract:

- Copy: destination, source, count, overlap behavior.
- Clear: destination, value, count.
- String operation: terminator, maximum length, output termination.
- Cookie: saved cookie, check before return, failure target.
- Stack probing: page touching for large local allocation.
- SEH: exception registration manipulation via FS:[0].
- Virtual/function pointer call: data controls code address.
- Import thunk: API boundary.

MSVC emits these patterns because it must implement ABI rules, security options, runtime checks, and optimized library behavior.

9.3 Common compiler patterns

- `rep stosd/stosb`: `memset`-like initialization.
- `rep movsd/movsb`: `memcpy`-like copy.
- `call @__security_check_cookie@4`: stack cookie verification.
- `xor eax, ebp` near prologue: cookie derivation.
- `call __chkstk`: stack probing for large frames.

- push offset handler; push fs:[0]; mov fs:[0],esp: x86 SEH registration.
- jmp dword ptr ds:__imp__API: import thunk.

9.4 Fully annotated example

```

sub_401800:
    push    ebp
    mov     ebp, esp
    sub     esp, 84h
    mov     eax, ___security_cookie
    xor     eax, ebp
    mov     [ebp-4], eax
    lea     edi, [ebp-84h]
    xor     eax, eax
    mov     ecx, 20h
    rep     stosd
    lea     eax, [ebp-84h]
    push    eax
    call    sub_402100
    add     esp, 4
    mov     ecx, [ebp-4]
    xor     ecx, ebp
    call    @__security_check_cookie@4
    mov     esp, ebp
    pop     ebp
    retn

```

Annotated:

```

mov     eax, ___security_cookie / xor     eax, ebp / mov     [ebp-4], eax
; Stack cookie stored in the frame. This is compiler-inserted protection,
↪ not
; programmer business logic.

lea     edi, [ebp-84h] / xor     eax, eax / mov     ecx, 20h / rep     stosd
; Clear 0x20 dwords = 0x80 bytes of local buffer/object.

call    sub_402100
; Programmer logic resumes: pass the cleared local object to another
↪ function.

mov     ecx, [ebp-4] / xor     ecx, ebp / call    __security_check_cookie
; Verify cookie before returning.

```

The compiler generated the cookie because the function has stack objects that qualify for protection. The programmer wrote code that needed a local object and a call using that object.

9.5 Reverse engineering thought process

Mark runtime artifacts in your notes:

```
compiler: frame, cookie setup/check, rep stosd clear
programmer: zero-initialized local object passed to sub_402100
unknown: what sub_402100 does with that object
```

An exploit developer does not ignore the cookie. They classify it: stack smash may be detected before RET, so exploit strategy may require one of these bypass approaches:

- **SEH overwrite:** if the function registers an SEH handler and the buffer overflow reaches the exception registration record, triggering an exception before the cookie check transfers control through the corrupted handler.
- **Cookie disclosure:** an information leak in a prior request or response reveals the cookie value, allowing the attacker to reconstruct the expected cookie in the overflow payload.
- **Write-what-where:** if the overflow corrupts a pointer or index used in a write before the function returns, the attacker gains an arbitrary write without needing to pass the cookie check.
- **Non-cookie-covered path:** the compiler does not protect every function. A different call site or a function without /GS protection may offer the same overflow without a cookie.
- **Heap or global corruption:** the vulnerable copy writes past the stack frame into adjacent memory, but the exploit targets heap metadata, a global function pointer, or a vtable rather than the saved return address.

9.6 Common mistakes

- Treating `__security_check_cookie` as an application validation function.
- Calling every `rep movs` safe or unsafe without count/destination analysis.
- Ignoring stack probing and misreading it as an application loop.
- Assuming import thunks are meaningful functions.

9.7 Exercises

```
sub_401870:
    push    ebp
    mov     ebp, esp
    mov     eax, 5000h
    call    __chkstk
    lea     eax, [ebp-5000h]
    push    eax
    call    sub_402300
    add     esp, 4
    mov     esp, ebp
    pop     ebp
    retn
```

Questions:

- Why does `__chkstk` exist?
- What programmer-level fact caused this pattern?
- What should you analyze next?

9.8 Challenge problems

```
push    offset handler
push    dword ptr fs:[0]
mov     dword ptr fs:[0], esp
...
mov     ecx, [esp]
mov     dword ptr fs:[0], ecx
add     esp, 8
```

Identify the pattern and explain how it affects exploit-oriented analysis.

9.9 Solutions with reasoning

Exercise solution:

On 32-bit MSVC, `__chkstk` is the compiler helper used for large stack allocations. The requested allocation size is placed in EAX, and the helper probes the guard pages while establishing the large stack frame. Do not add a second `sub esp, eax` after this pattern unless you have verified the exact helper convention in the target compiler/runtime. The programmer-level fact is a large local object or `alloca`-like allocation around `0x5000` bytes. A real build with this allocation size would likely also include a stack cookie; it is omitted here to focus on the probing pattern. Analyze `sub_402300` and all writes to the large local buffer; the probe itself is not domain logic.

Plausible pseudocode:

```
void f(void) {
    unsigned char big[0x5000];
    sub_402300(big);
}
```

Challenge solution:

This is x86 Structured Exception Handling registration. The function links an exception registration record through `FS:[0]`, then restores the previous head. For exploit analysis, SEH matters because stack corruption may overwrite handler records, and exception paths can transfer control before normal epilogue/cookie logic depending on layout and compiler options.

Chapter 10

Methodology: From Unknown Function to Working Model

10.1 Learning objectives

- Analyze an unknown function with a repeatable process.
- Track origin, destination, control conditions, and invariants.
- Separate facts, hypotheses, and unresolved questions.
- Know when to switch from IDA/Ghidra to WinDbg.

10.2 Concept discussion

Good reverse engineering is disciplined uncertainty reduction. You do not need to understand every instruction at once. You need to identify the load-bearing facts and test the model.

Use this method:

1. Find boundaries: prologue, epilogue, tail calls, exception exits.
2. Identify inputs: arguments, globals, object fields, imports, prior calls.
3. Identify outputs: return value, writes, calls, global changes.
4. Recover guards: null checks, bounds checks, auth checks, type checks.
5. Track attacker-controlled values.
6. Identify invariants after each guard.
7. Classify compiler artifacts.
8. Draft pseudocode only after the fact ledger stabilizes.
9. Verify uncertain facts in WinDbg.

10.3 Common compiler patterns

- Repeated return `-1` blocks: validation failure.
- Calls one layer deeper: wrappers hiding real behavior.
- Status codes checked with `test eax, eax` or `cmp eax, 0`.
- Auth/session fields checked before parser calls.

- Logging calls that use attacker data but do not consume it dangerously.

10.4 Fully annotated example

```

sub_401900:
    push    ebp
    mov     ebp, esp
    push    esi
    mov     esi, [ebp+8]
    cmp     byte ptr [esi+20h], 0
    jz      noauth
    push    [ebp+10h]
    push    [ebp+0Ch]
    call    parse_config_set
    add     esp, 8
    test    eax, eax
    jnz     bad
    push    offset ok_msg
    push    esi
    call    send_status
    add     esp, 8
    pop     esi
    pop     ebp
    retn

noauth:
    push    offset auth_msg
    push    esi
    call    send_status
    add     esp, 8
    pop     esi
    pop     ebp
    retn

bad:
    push    offset bad_msg
    push    esi
    call    send_status
    add     esp, 8
    pop     esi
    pop     ebp
    retn

```

Annotated:

```

mov esi, [ebp+8]
cmp byte ptr [esi+20h], 0 / jz noauth
; Session/client object has an auth-like byte at +0x20. It gates parser
↪ access.

```



```
push [ebp+10h] / push [ebp+0Ch] / call parse_config_set
; Body pointer and body length are passed one layer deeper. The bug, if
  ↳ any, is
; probably not in this wrapper.

test eax, eax / jnz bad
; Parser status controls response.
```

The CPU is answering: “Is this client authenticated, and did the parser accept the body?” The programmer wrote workflow control. An exploit developer follows the attacker-controlled body into `parse_config_set`.

10.5 Reverse engineering thought process

Fact ledger:

```
arg0 = client/session pointer
field +0x20 gates parser
arg1/arg2 = parser inputs
parse_config_set returns zero on success
send_status consumes result messages
```

Open questions:

```
What writes field +0x20?
Where do arg1/arg2 originate?
Does parse_config_set copy without bounds?
Can auth be bypassed or legitimately obtained?
```

10.6 Common mistakes

- Stopping at the handler and declaring it safe.
- Treating auth gates as irrelevant because they are not memory corruption.
- Spending time on response formatting before following input data.
- Writing pseudocode before knowing which arguments are body pointer and length.

10.7 Exercises

```
sub_401980:
    mov     eax, [esp+4]
    cmp     dword ptr [eax+14h], 3
    jnz     short reject
    push    [esp+0Ch]
    push    [esp+0Ch]
```

```
    call    sub_402500
    add     esp, 8
    retn
reject:
    mov     eax, 0FFFFFFFFh
    retn
```

Questions:

- What is the gate?
- What value flows past the gate?
- What must be verified before trusting the stack offsets?

10.8 Challenge problems

Given a function with one obvious unsafe memcpy, list the evidence you need before calling it exploitable.

10.9 Solutions with reasoning

Exercise solution:

The gate is a field comparison at object offset +0x14 against value 3. Arguments after the gate are forwarded into sub_402500, but the repeated [esp+0Ch] is suspicious because pushes change ESP. Verify actual arguments by normalizing stack deltas in IDA or breaking before the call and dumping the stack. The source-level model is “only state 3 may call worker.”

Challenge solution:

You need destination location and size, copy count, source control, reachability, mitigations, overwrite target, and post-overwrite control flow. Unsafe-looking copy is not enough. A bounded copy into a large buffer may be safe; an unbounded copy may be unreachable; an exploitable copy must connect attacker bytes to a security-relevant overwrite under reachable conditions.

Chapter 11

Exploit-Oriented Reading

11.1 Learning objectives

- Recognize exploit-relevant bug classes in assembly.
- Connect semantic recovery to attacker control.
- Identify stack, heap, integer, format string, lifetime, SEH, DEP, and ROP implications.
- Avoid declaring exploitability from one instruction.

11.2 Concept discussion

Exploit-oriented reverse engineering is not “find scary functions.” It is reachability plus control plus impact.

For each suspicious operation, ask:

- Can attacker-controlled data reach this operation?
- Is the destination bounded by the same value being copied?
- Is there a mismatch between length, capacity, signedness, or units?
- What object lies after the overwritten region?
- Does normal control flow reach an exploitable use?
- Do mitigations change the strategy?

11.3 Common compiler patterns

- Stack overflow: local buffer address passed as destination with unchecked count.
- Heap overflow: heap pointer plus attacker length, no capacity check.
- Integer overflow: `add/impl` before allocation or copy, no carry/range check.
- Integer truncation: `movzx eax, ax` or `and eax, 0FFFFh` after arithmetic silently drops high bits; a 32-bit length that passes a check can become a smaller value used as a copy count. Watch for `movzx/movsx` between an arithmetic result and its use as a size, and for signed/unsigned mismatch where `j l` guards a value later consumed by an unsigned `rep movs` count.
- Format string: attacker pointer passed as format argument.

- UAF: `_free(ptr)` followed by later dereference through alias.
- Double free: same pointer freed on multiple paths without nulling all aliases.
- Off-by-one: check `len <= cap` followed by `dst[len] = 0`.
- SEH: stack exception records near corruptible buffers.
- DEP: indirect calls to `VirtualProtect`, `VirtualAlloc`, or ROP-relevant imports.
- ROP-friendly code: controllable dispatch, stack pivot gadgets, imported API pointers.

11.4 Fully annotated example

```
sub_401A00:
    push    ebp
    mov     ebp, esp
    sub     esp, 88h
    lea     eax, [ebp-80h]
    push    [ebp+0Ch]
    push    [ebp+8]
    push    eax
    call    _memcpy
    add     esp, 0Ch
    mov     eax, 1
    mov     esp, ebp
    pop     ebp
    ret     4
```

Annotated:

```
sub esp, 88h / lea eax, [ebp-80h]
; Local destination buffer appears to be 128 bytes.

push [ebp+0Ch]
; Count comes from caller.

push [ebp+8]
; Source comes from caller.

call _memcpy
; Raw count-based copy. No local bounds check visible.
```

This is a stack-overflow candidate. It is not proven exploitable yet. You still need caller provenance, maximum count, cookie presence, saved return offset, SEH layout, and post-copy behavior.

11.5 Reverse engineering thought process

Exploit ledger:

```
destination: stack local at ebp-0x80
capacity hypothesis: 0x80 bytes
```

```
source: caller-controlled? unknown
count: caller-controlled? unknown
check: none in this function
mitigation observed: no cookie in snippet
impact: possible overwrite above local buffer
next proof: xrefs and debugger crash offset
```

11.6 Common mistakes

- Calling memcpy itself a vulnerability.
- Ignoring caller-side validation.
- Missing integer unit mismatch: elements vs bytes.
- Assuming a crash means EIP control.
- Treating DEP code as irrelevant because the current bug is a stack overflow.

11.7 Exercises

```
sub_401A80:
    mov     eax, [esp+8]
    add     eax, 10h
    push    eax
    call    _malloc
    add     esp, 4
    mov     ecx, [esp+8]
    shl     ecx, 4
    push    ecx
    push    [esp+8]
    push    eax
    call    _memcpy
    add     esp, 0Ch
    retn
```

Questions:

- What arithmetic mismatch may exist?
- Which values need provenance?
- What bug class is plausible?

11.8 Challenge problems

```
sub_401AD0:
    push    [esp+4]
    call    _printf
    add     esp, 4
```

retn

Why is this more dangerous than `printf("%s", user)` even though both print user data?

11.9 Solutions with reasoning

Exercise solution:

Allocation size is `n + 0x10`, but copy size is `n << 4`. For large or even moderate `n`, copy size can exceed allocation size. There may also be integer overflow in either expression. Provenance is needed for `n` and source pointer. The plausible class is heap overflow via size-calculation mismatch.

Plausible pseudocode:

```
buf = malloc(n + 0x10);  
memcpy(buf, src, n * 16);
```

Challenge solution:

The user pointer is passed as the format string, not as data for a fixed format. That allows `%x`, `%s`, `%n`, positional parameters, and width specifiers to change reads and writes performed by `printf`. The programmer assumed the string was inert text; the C runtime treats it as a command language.

Chapter 12

Case Studies from Vulnerable Windows Service Patterns

12.1 Learning objectives

- Apply the full methodology to realistic service-handler shapes.
- Follow attacker-controlled network data through dispatch, auth, parsers, and copy helpers.
- Distinguish suspicious-safe code from exploitable code.
- Build an exploit-relevant model without relying on source names.

12.2 Concept discussion

Real vulnerable services rarely put the bug in the first function you open. The dispatcher routes by opcode. The handler checks auth. The parser extracts fields. The helper copies bytes. The bug appears when a programmer assumption at one layer is not enforced at the layer doing memory writes.

These case studies are modeled on patterns found in the repository's Windows x86 training labs under `Tools/` and `scripts/`. The function names below are representative; the actual lab binaries use similar shapes:

- opcode dispatch similar to `dispatch_command`
- safe copy helpers similar to `copy_metadata` and `copy_echo_payload`
- unsafe record copy similar to `copy_record_payload`
- auth-gated parser routing similar to config-set handlers

12.3 Common compiler patterns

- `strcmp` dispatch: compare command prefixes, branch to handlers.
- Switch dispatch: opcode range check and jump table.
- Safe copy helper: `if (len >= dst_size) fail; memcpy; dst[len]=0.`
- Unsafe copy helper: local buffer plus attacker-controlled count.
- Auth gate: session field checked before parser call.

- Response/logging calls: noise unless they consume attacker data dangerously.

12.4 Fully annotated example

Assembly first:

```
sub_402000:
    push    ebp
    mov     ebp, esp
    sub     esp, 84h
    mov     eax, [ebp+0Ch]
    cmp     eax, [ebp+10h]
    ja      reject
    lea     ecx, [ebp-80h]
    push    eax
    push    [ebp+8]
    push    ecx
    call    _memcpy
    add     esp, 0Ch
    xor     eax, eax
    mov     esp, ebp
    pop     ebp
    retn

reject:
    mov     eax, 0FFFFFFFFh
    mov     esp, ebp
    pop     ebp
    retn
```

Annotated:

```
sub esp,84h / lea ecx,[ebp-80h]
; Local stack buffer around 128 bytes.

mov eax,[ebp+0Ch]
cmp eax,[ebp+10h] / ja reject
; Checks copy_len <= source_len or declared_len. This is not a destination
; capacity check.

push eax / push [ebp+8] / push ecx / call _memcpy
; Copy attacker-controlled count into fixed local destination if prior
; ↪ length
; relation holds.
```

The key semantic failure: the code proves the copy does not read past the source record, but it does not prove the local destination can hold the copy. The programmer assumed “copy length is valid” meant “copy length is safe for every destination.” The compiler merely implemented that assumption.

12.5 Reverse engineering thought process

Case-study ledger:

```
data origin: network body, through parser
gate: copy_len <= source_len
missing invariant: copy_len < sizeof(local_buffer)
destination: stack local
impact: overwrite frame if copy_len > 0x80
next steps: identify reachability, saved return offset, cookie/SEH, bad
↳ chars,
DEP strategy
```

A reverse engineer explains the bug as an invariant mismatch. An exploit developer turns the mismatch into control: offset, controllable bytes, reliable return path, and mitigation bypass.

12.6 Common mistakes

- Seeing a length check and declaring the copy safe.
- Ignoring which buffer the length check protects.
- Stopping at opcode dispatch instead of following the body pointer.
- Treating auth gates as blockers instead of reachability conditions.
- Confusing source length, requested copy length, and destination capacity.

12.7 Exercises

Assembly only:

```
sub_402080:
    push    ebp
    mov     ebp, esp
    sub     esp, 40h
    mov     eax, [ebp+8]
    cmp     byte ptr [eax+24h], 0
    jz      noauth
    lea     ecx, [ebp-40h]
    push    [ebp+10h]
    push    [ebp+0Ch]
    push    ecx
    call    parse_set_line
    add     esp, 0Ch
    test    eax, eax
    jnz     bad
    xor     eax, eax
    mov     esp, ebp
    pop     ebp
    ret     0
```

```
noauth:
    mov     eax, 0FFFFFFEh
    mov     esp, ebp
    pop     ebp
    retn

bad:
    mov     eax, 0FFFFFFFh
    mov     esp, ebp
    pop     ebp
    retn
```

Questions:

- What must be true before parser execution?
- What is the likely role of the local buffer passed to parse_set_line?
- What exploit question does the third argument raise?
- Where should you go next?

12.8 Challenge problems

Assembly only:

```
sub_402100:
    push    ebp
    mov     ebp, esp
    sub     esp, 48h
    lea     eax, [ebp-40h]
    push    eax
    push    offset fmt_get
    push    [ebp+8]
    call    _scanf
    add     esp, 0Ch
    cmp     eax, 1
    jnz     fail
    lea     eax, [ebp-40h]
    push    eax
    call    config_lookup
    add     esp, 4
    pop     ebp
    retn

fail:
    mov     eax, 0FFFFFFFh
    pop     ebp
    retn
```

Assume `fmt_get` may be either `"get %63s"` or `"get %s"`. Explain how the security conclusion changes and how to prove which one is present.

12.9 Solutions with reasoning

Exercise solution:

The session/client field at offset +0x24 must be nonzero before the parser runs. That is likely an auth or state gate. The local buffer at `ebp-0x40` (64 bytes) is passed as the first argument to `parse_set_line`, likely as a destination for parsed output. The second argument is `[ebp+0Ch]` (caller body pointer) and the third is `[ebp+10h]` (caller length or size). The exploit question is whether `parse_set_line` can write more than 64 bytes into the local buffer based on the third argument. The next target is `parse_set_line`, because this wrapper only gates and forwards.

Plausible pseudocode:

```
int handle_set(Client *c, const char *body, unsigned len) {
    char local[64];
    if (!c->authenticated) return -2;
    if (parse_set_line(local, body, len) != 0) return -1;
    return 0;
}
```

Challenge solution:

If the format is "get %63s", `_sscanf` writes at most 63 non-whitespace bytes plus a terminator into a 64-byte buffer, which matches the local buffer size. If the format is "get %s", the copy is unbounded and can overflow the local buffer. Prove it by following the `fmt_get` data reference in IDA/Ghidra or by dumping the address in WinDbg with `da fmt_get`. The security conclusion depends on the format string, not on the presence of `_sscanf` alone.

12.10 Final capstone

Analyze this complete function as if it were unknown:

```
sub_402200:
    push    ebp
    mov     ebp, esp
    sub     esp, 0CCh
    push    esi
    mov     esi, [ebp+8]
    test    esi, esi
    jz      fail
    cmp     byte ptr [esi+20h], 0
    jz      fail
    mov     eax, [ebp+10h]
    cmp     eax, [ebp+14h]
    ja      fail
    lea     ecx, [ebp-80h]
    push    eax
    push    [ebp+0Ch]
    push    ecx
    call    _memcpy
```

```
add     esp, 0Ch
lea     eax, [ebp-80h]
push    eax
call    process_record
add     esp, 4
xor     eax, eax
pop     esi
mov     esp, ebp
pop     ebp
retn

fail:
mov     eax, 0FFFFFFFh
pop     esi
mov     esp, ebp
pop     ebp
retn
```

Solution:

Facts:

- `arg0` is a pointer, preserved in ESI.
- `arg0` must be non-null.
- Byte field `arg0+0x20` must be nonzero before processing.
- `arg2 <= arg3` unsigned is required before the copy.
- Destination is a stack local at `ebp-0x80`.
- Source is `arg1`.
- Count is `arg2`.
- The local buffer is passed to `process_record`.

Interpretation:

This is an auth/state-gated record-processing wrapper. It checks that the copy length does not exceed some source/declaration length, copies into a fixed stack buffer, then processes the local copy.

Exploit reasoning:

The length check does not mention the destination capacity. If `arg2` can exceed `0x80`, this is a stack-overflow candidate even though a length check exists. If a stack cookie is absent, saved return address or saved registers may be reachable. If a cookie exists in a real build, look for SEH layout, non-return control flow, adjacent object corruption, or a different bug path.

Plausible pseudocode after reasoning:

```
int handle_record(Client *c, const unsigned char *body,
                  unsigned copy_len, unsigned declared_len) {
    unsigned char local[128];

    if (!c) return -1;
    if (!c->authenticated) return -1;
```

```
if (copy_len > declared_len) return -1;

memcpy(local, body, copy_len);
process_record(local);
return 0;
}
```

The reverse engineer's final model is not "memcpy overflow." It is: an authenticated record handler validates source bounds but fails to validate destination capacity before copying network-controlled bytes into a stack local.

12.11 Appendix A: Exercise answering template

Use this before pseudocode:

Function boundary:
Inputs:
Outputs:
Memory reads:
Memory writes:
Calls:
Branch questions:
Value ranges after checks:
Compiler artifacts:
Programmer assumptions:
Attacker-controlled values:
Exploit relevance:
Unresolved facts to verify:

12.12 Appendix B: Fast semantic checklist

- test p,p before [p+off]: null guard.
- movzx: unsigned narrow value.
- movsx: signed narrow value.
- ja/jb: unsigned comparison.
- jg/jl: signed comparison.
- lea without later dereference: arithmetic.
- lea local; push local; call: address-taken local.
- dst[len]=0: string termination and possible off-by-one.
- len < cap: usually leaves room for terminator.
- len <= cap: dangerous if terminator is also written.
- call [reg+off]: code pointer read from data.
- jmp __imp_*: import thunk.
- __security_check_cookie: compiler protection, not business logic.

- `__chkstk`: large stack allocation, not application loop.

12.13 Appendix C: WinDbg verification prompts

Use WinDbg to promote hypotheses to facts:

```
bp module!function
uf module!function
dd esp L20
dd ebp-100 L80
da poi(esp+4)
dds poi(table) L10
ba w4 address
!exchain
kv
```

Verify:

- actual argument order at call sites
- whether source bytes are attacker-controlled
- exact overwrite offset
- stack cookie placement
- SEH chain position
- imported API addresses
- whether a suspicious path is reachable

Chapter 13

Printable Worksheets and Checklists

Use these pages while working through unknown functions. They are intentionally repetitive. The repetition builds the habit that matters in real reversing: facts first, source reconstruction second.

13.0.1 Unknown function worksheet

Function address/name:

Caller(s):

Callee(s):

Boundary:

Prologue:

Epilogue:

Tail calls:

Exception exits:

Incoming values:

☐ stack arguments

☐ register arguments

☐ globals

☐ object fields

☐ imported/runtime state

Outgoing values:

☐ EAX return

☐ memory writes

☐ global writes

☐ calls with modified arguments

☐ indirect control flow

Compiler scaffolding:

☐ saved registers

☐ frame allocation

☐ stack cookie

- ☐ stack probe
- ☐ SEH registration
- ☐ import thunk

One-sentence current hypothesis:

Evidence that supports it:

Evidence that contradicts or weakens it:

Next verification step:

13.0.2 Branch worksheet

Compare/test instruction:

Conditional jump:

Signed or unsigned:

Taken edge means:

Fall-through edge means:

Value constrained:

Constraint after taken edge:

Constraint after fall-through edge:

Is this a validation gate?

- ☐ null check
- ☐ type/opcode check
- ☐ length/capacity check
- ☐ auth/state check
- ☐ error/status check
- ☐ other

What becomes safe after this branch?

What is still not proven?

13.0.3 Call-site worksheet

Call target:

Direct or indirect:

Caller cleanup or callee cleanup:

Arguments:

Note: `arg0` is the first C parameter, usually `[ebp+8]`; it is pushed last
↪ at a `cdecl/stdcall` call site.

`arg0`:


```
arg1:
arg2:
arg3:
arg4:
```

For each pointer argument:

```
Origin:
Points to:
Read or write:
Size/capacity:
Attacker-controlled:
```

Return value use:

```
[ ] ignored
[ ] zero/nonzero check
[ ] signed error check
[ ] pointer dereference
[ ] length/count
[ ] stored for later
```

Question this call answers:

13.0.4 Pointer provenance and alias worksheet

Pointer value:

First observed at:

Original source:

```
[ ] stack argument
[ ] register argument
[ ] global
[ ] heap allocation
[ ] object field
[ ] return value
[ ] computed address
```

Copies/aliases:

```
register alias 1:
register alias 2:
stack spill:
structure field:
global/table entry:
```

Adjustments:

```
added offset:
scaled index:
pointer increment:
cast or width change:
```

Dereferences:

read at:
write at:
width:

Calls that may clobber aliases:

caller-saved registers live across call:
preserved registers:
memory aliases still valid:

Trust:

attacker-controlled pointer:
attacker-controlled pointed-to data:
validated before dereference:
validated before write:

Unresolved alias questions:

13.0.5 Copy-operation worksheet

Operation:

☐ memcpy-like
☐ memmove-like
☐ strcpy-like
☐ strncpy-like
☐ memset-like
☐ rep movs/stos
☐ sscanf/string parser

Destination:

Address:
Storage:
☐ stack
☐ heap
☐ global
☐ caller-provided
Capacity:
How capacity is known:

Source:

Address:
Origin:
Attacker-controlled:

Count/termination:

Count value:

Count origin:

Terminator written:

Maximum proven:

Safety relationship:

[] count <= destination capacity

[] count < destination capacity because terminator follows

[] only source length checked

[] no destination check found

[] unknown

Exploit notes:

13.0.6 Structure recovery worksheet

Base pointer:

How base pointer is obtained:

Observed fields:

+0x00:

+0x04:

+0x08:

+0x0C:

+0x10:

+0x14:

+0x18:

+0x1C:

+0x20:

For each field:

Read/write:

Width:

Signedness:

Compared against:

Passed to:

Stored from:

Likely object role:

Invariants:

Trust boundary:

13.0.7 Loop worksheet

Loop header:

Loop body:

Exit block:

Induction variable:

Initial value:

Update:

Exit condition:

Memory access pattern:

Element size:

Base pointer:

Index or pointer bump:

Loop type:

☐ counted

☐ sentinel

☐ bounded sentinel

☐ traversal

☐ retry/error loop

Invariant before each iteration:

What is returned or written:

13.0.8 Dispatch and switch worksheet

Dispatch function address:

Dispatch key register/operand:

Key source (argument, field, computed):

Range check:

Instruction:

Valid range:

Unsigned or signed:

Default/out-of-range target:

Dispatch mechanism:

☐ compare chain

☐ jump table

☐ lookup table + indirect call

☐ tail call series

☐ other

Targets:

case 0:

case 1:

case 2:

case 3:

default:

Which target receives attacker-controlled data:
Which target performs memory writes:
Are any targets tail calls (jmp, not call):
Is the dispatch key attacker-controlled:

Next analysis target:

13.0.9 Exploit triage worksheet

Suspicious operation:
Reachability:
Attacker-controlled input:
Validation gates:
Missing invariant:

Bug class candidate:
[] stack overflow
[] heap overflow
[] integer overflow/truncation
[] format string
[] use-after-free
[] double free
[] off-by-one
[] SEH overwrite
[] function pointer/vtable corruption
[] info leak

Overwrite/read target:
Offset calculation:
Bad characters or input constraints:
Mitigations:
[] stack cookie
[] DEP/NX
[] ASLR
[] SEHOP/SafeSEH
[] heap hardening

What would prove exploitability:
What would disprove exploitability:

13.0.10 WinDbg verification worksheet

Breakpoint:
Input used:

Before call:

esp:
ebp:
eax:
ecx:
edx:
esi:
edi:

Stack dump:

Memory dump of destination:

Memory dump of source:

After call:

eax:
changed memory:
exception:

Static hypothesis confirmed?

☐ yes
☐ no
☐ partially

Correction to static model:

13.0.11 Shellcode analysis worksheet

Sample source:

Entry point:

EIP recovery method:

☐ call/pop
☐ jmp/call backward
☐ fstenv
☐ other

Register holding recovered EIP:

Encoder/decoder:

☐ single-byte XOR
☐ additive
☐ multi-byte key
☐ sub-based
☐ none / cleartext

Key value:

Encoded region start:

Encoded region length:

Bad characters avoided:

PEB walk present:

☐ yes

☐ no

Module list used:

☐ InLoadOrderModuleList (+0Ch)

☐ InMemoryOrderModuleList (+14h)

☐ InInitializationOrderModuleList (+1Ch)

Target module (by position):

DllBase offset:

API resolution:

☐ hash-based (ROR/ADD loop)

☐ string comparison

☐ GetProcAddress after resolution

☐ hardcoded addresses

Hash algorithm:

APIs resolved:

Payload action:

☐ reverse shell

☐ bind shell

☐ download-and-execute

☐ command execution

☐ other

Null-byte avoidance techniques observed:

WinDbg verification:

Break at decoder end, dump decoded payload:

Break at API call, verify arguments:

13.0.12 ROP chain worksheet

Overflow target:

☐ saved return address

☐ SEH handler

☐ function pointer

☐ vtable entry

Stack pivot (if needed):

Pivot gadget address:

Pivot instruction:

Controlled register:

New ESP value:

Chain trace:

Slot	Address	Gadget/Value	Effect
+00:			
+04:			
+08:			
+0C:			
+10:			
+14:			
+18:			
+1C:			
+20:			
+24:			
+28:			
+2C:			
+30:			

Register state after chain:

EAX:
EBX:
ECX:
EDX:
ESI:
EDI:
ESP:
EBP:

API call being set up:

Function:
arg0 (return address after API):
arg1:
arg2:
arg3:
arg4:

DEP bypass method:

[] VirtualProtect
[] VirtualAlloc
[] WriteProcessMemory
[] SetProcessDEPPolicy
[] NtSetInformationProcess
[] other

Gadget quality notes:

Non-ASLR module used:
Clobbered registers:
Conditional branches in gadgets:
Bad characters in addresses:

What would break the chain:

13.0.13 Daily practice drill

Use this for one unknown function per day.

Function:

Timebox: 25 minutes

Minute 0–5:

Boundary, arguments, return, obvious calls.

Minute 5–10:

Branch questions and validated ranges.

Minute 10–15:

Data origin/destination and memory writes.

Minute 15–20:

Programmer assumptions and exploit relevance.

Minute 20–25:

Pseudocode and verification plan.

Final sentence:

This function exists to...

13.0.14 Chapter review grid

Chapter:

Three patterns I can recognize:

- 1.
- 2.
- 3.

One pattern I can derive but not yet recognize quickly:

One mistake I made:

One WinDbg command that would verify the main idea:

One exploit question raised by the chapter:

Chapter 14

Shellcode Idioms and Position-Independent Patterns

14.1 Learning objectives

- Recognize position-independent code patterns in x86 shellcode.
- Identify `call/pop` and `fstenv`-based EIP recovery.
- Read PEB-walking sequences that resolve API addresses at runtime.
- Recognize encoder/decoder stubs and egghunter patterns.
- Distinguish shellcode idioms from normal compiler output.

14.2 Concept discussion

Shellcode runs without the loader's help. It cannot use import tables, fixed addresses, or relocations. Every address it needs must be derived at runtime. That constraint produces distinctive assembly patterns a reverse engineer should recognize on sight.

The core problems shellcode solves:

- **Where am I?** The code must learn its own address to reference embedded data (strings, hashes, encoded payload).
- **Where are the APIs?** Without an import table the code must walk the PEB to find loaded modules and resolve exports by name or hash.
- **How do I avoid bad characters?** Network protocols, string functions, and copy operations may filter bytes like `0x00`, `0x0A`, `0x0D`, or `0x80+`. Encoder stubs transform the payload to survive transit, then decode in place.
- **Where is my payload?** Staged shellcode or egghunters search memory for a marker tag to locate a larger payload placed elsewhere.

These are not compiler patterns. They are hand-written or tool-generated sequences with a distinctive shape.

14.3 Common shellcode patterns

- `call next; next: pop reg`: classic EIP recovery.
- `jmp over; tag: call back; over: ...`: call-backward variant for data reference.
- `fstenv [esp-0Ch]` followed by `pop reg`: FPU-based EIP recovery.
- `mov eax, fs:[30h]`: PEB access.
- `mov eax, [eax+0Ch]`: PEB->Ldr.
- `mov eax, [eax+14h]`: `InMemoryOrderModuleList`.
- `ror edx, 0Dh; add edx, [esi]`: hash accumulation for API name resolution.
- `xor byte ptr [edi], key; inc edi; loop`: single-byte XOR decoder.
- `cmp dword ptr [edi], tag; jz found; inc edi`: egghunter linear scan.
- Null-byte avoidance: `xor eax, eax` instead of `mov eax, 0`; push byte 1 instead of push 1; `sub eax, -value` instead of `add eax, value`.

14.4 Fully annotated example

14.4.1 EIP recovery and data reference

```

shellcode_start:
    jmp     short get_data
callback:
    pop     esi
    ; ESI now points to the string below.
    xor     eax, eax
    mov     byte ptr [esi+0Dh], al
    push    eax
    push    esi
    call    ebp
    ; Assume EBP holds address of a resolved API.
get_data:
    call    callback
    db      "Hello, world!", 0FFh

```

Annotated:

```

jmp short get_data
; Skip over the callback body to reach the CALL below.

call callback
; CALL pushes the address of the next byte (the string) onto the stack, then
; jumps backward. This is not a normal function call. The return address IS
; the
; data pointer.

pop esi
; Pop the pushed return address. ESI now holds the runtime address of the
; embedded string.

```

```

mov byte ptr [esi+0Dh], al
; Overwrite the placeholder byte after "Hello, world!" with a NUL
  ⇨ terminator.
; The 0xFF placeholder avoided a null byte in the shellcode body.

```

The programmer wrote position-independent data referencing. A compiler would never emit `call` to push a data address. Seeing `call` immediately followed by non-code bytes or a `pop` into a general-purpose register is a strong shellcode indicator.

14.4.2 PEB walk for module resolution

```

xor     eax, eax
mov     eax, fs:[eax+30h]
mov     eax, [eax+0Ch]
mov     esi, [eax+14h]
next_module:
  lodsb
  ; Actually: lodsd (load dword from [esi], advance esi)
  ; Corrected below.

```

Corrected and annotated PEB walk:

```

xor     eax, eax
mov     eax, fs:[eax+30h]
; Read the PEB pointer from the TEB. Using [eax+30h] instead of fs:[30h]
; avoids a null byte in the displacement encoding.

mov     eax, [eax+0Ch]
; PEB->Ldr (PEB_LDR_DATA pointer).

mov     esi, [eax+14h]
; Ldr->InMemoryOrderModuleList.Flink. This is a LIST_ENTRY pointing to
  ⇨ the
; first loaded module's LDR_DATA_TABLE_ENTRY (at the InMemoryOrderLinks
; offset).

next_module:
  lodsd
  ; Load [ESI] into EAX and advance ESI by 4. EAX now holds the Flink of
  ⇨ the
  ; next list entry.

  xchg   esi, eax
  ; Move to the next entry. ESI = new list node.

  mov     ebx, [esi+10h]

```

```
; DllBase of this module (base address of the loaded image).
```

The reverse engineer sees `fs:[30h]` and knows this is PEB access. The `0Ch` → `14h` → `lodsd` chain is the standard module-list walk. Variations exist with `InLoadOrderModuleList` (offset `0Ch` instead of `14h`) or `InInitializationOrderModuleList` (offset `1Ch`).

14.5 Reverse engineering thought process

When you encounter code that does not look like compiler output, ask:

```
EIP recovery method: call/pop, fstenv, or other
Data references: relative to recovered EIP?
PEB access: fs:[30h] present?
Module walk: which list order?
API resolution: hash-based or string-based?
Encoder: XOR, ADD, SUB, or multi-byte?
Egg tag: 4-byte or 8-byte marker?
Null avoidance: which techniques?
```

Shellcode is sequential: it does not have the call graph of compiled code. Walk it linearly and mark each phase boundary (decoder, resolver, payload).

14.6 Common mistakes

- Treating `call/pop` as a function call and trying to analyze the “callee.”
- Missing `fstenv`-based EIP recovery because FPU instructions look unrelated.
- Assuming `fs:[30h]` is SEH-related. SEH uses `fs:[0]`, not `fs:[30h]`.
- Stopping at the decoder stub and missing the real payload underneath.
- Treating encoded bytes as data when they are code that has not been decoded yet.
- Confusing the egg tag with a magic constant or structure signature.

14.7 Exercises

```
xor    ecx, ecx
mov    ecx, fs:[ecx+30h]
mov    ecx, [ecx+0Ch]
mov    ecx, [ecx+14h]
mov    ecx, [ecx]
mov    ecx, [ecx]
mov    ebx, [ecx+10h]
```

Questions:

- What data structure is being traversed?
- Why does `mov ecx, [ecx]` appear twice?
- What does EBX hold at the end?

- Which module does this target and why?

14.8 Challenge problems

```
decoder:
    mov     edi, esi
    xor     ecx, ecx
    mov     cl, 38h
decode_loop:
    xor     byte ptr [edi], 0AAh
    inc     edi
    loop    decode_loop
    jmp     esi
```

Assume ESI was set by a prior call/pop. Explain:

- What transformation is applied?
- How many bytes are decoded?
- Why does execution jump to ESI after decoding?
- What must be true about the encoded payload for this to work?
- How would you extract the decoded payload statically?

14.9 Solutions with reasoning

Exercise solution:

The sequence walks the PEB module list. `fs:[ecx+30h]` reads the PEB pointer (using `ecx` as a zeroed base to avoid null bytes). `[PEB+0Ch]` is `Ldr`. `[Ldr+14h]` is `InMemoryOrderModuleList.Flink`, pointing to the first module entry. Each `mov ecx, [ecx]` follows the `Flink` to the next entry, so two iterations skip past the first two modules. On Windows, the standard load order in `InMemoryOrderModuleList` is:

1. The executable itself
2. `ntdll.dll`
3. `kernel32.dll`

Two `Flink` traversals reach the third entry, which is typically `kernel32.dll`. `[ecx+10h]` reads the `DllBase` field, so `EBX` holds the base address of `kernel32.dll`. This is the setup for export-table walking to resolve API functions like `WinExec`, `LoadLibraryA`, or `GetProcAddress`.

Challenge solution:

The decoder XORs each byte at `[edi]` with `0xAA` for `0x38` (56) bytes starting at the address in `ESI`. The `loop` instruction decrements `ECX` and jumps while nonzero. After decoding, `jmp esi` transfers execution to the now-decoded payload. The decoder is a single-byte XOR stub.

For this to work, the encoded payload must not contain the byte `0xAA` XORed with any of the decoder's own bytes (or the decoder must be placed outside the decoded region, which it is here).

since ESI points past the decoder). The payload also must not contain bytes that, after XOR with 0xAA, produce characters filtered by the delivery channel.

To extract the decoded payload statically, XOR each of the 56 bytes starting at the address after the `jmp esi` with 0xAA. In Python:

```
decoded = bytes(b ^ 0xAA for b in encoded_payload[:0x38])
```

Or in WinDbg, break at the `jmp esi` instruction and dump the decoded payload with `db esi L38`.

Chapter 15

ROP Gadgets and Code-Reuse Patterns

15.1 Learning objectives

- Read individual ROP gadgets as primitive operations on registers and the stack.
- Identify stack pivot gadgets and explain why they matter.
- Recognize mid-instruction gadgets and unintended instruction boundaries.
- Trace a short ROP chain through the stack to recover its intent.
- Connect gadget analysis to the broader exploit strategy (DEP bypass, API calls).

15.2 Concept discussion

When DEP prevents execution of attacker-supplied code on the stack or heap, the attacker reuses fragments of existing executable code. Each fragment ends with `ret` (or `ret N`), which pops the next address from the stack and transfers control there. By chaining these fragments through a controlled stack, the attacker builds arbitrary computation from existing code.

A ROP gadget is a short instruction sequence ending in `ret`. The reverse engineer reads gadgets differently from normal functions:

- There is no calling convention. The stack is the program.
- Each gadget consumes stack slots: one for the return address, plus any that `pop` instructions or `ret N` consume.
- Side effects on registers and memory are the gadget's purpose.
- The gadget may start at an aligned instruction boundary (intended gadget) or at an offset within a longer instruction (unintended/mid-instruction gadget).

15.3 Common gadget patterns

15.3.1 Register control

- `pop eax; ret`: load EAX from the stack. Consumes one slot for the value and one for the next gadget address.
- `pop ecx; pop ecx; ret`: load two values (or skip one and load one).
- `xchg eax, esp; ret`: dangerous stack pivot if EAX is controlled.

- `mov eax, ecx; ret`: transfer between registers.

15.3.2 Memory operations

- `mov [eax], ecx; ret`: write-what-where primitive.
- `mov [ecx], eax; ret`: write-what-where (reversed operands).
- `mov eax, [eax]; ret`: dereference and load.
- `add [eax], ecx; ret`: additive write primitive.
- `inc dword ptr [eax]; ret`: increment primitive.

15.3.3 Arithmetic and flag manipulation

- `add eax, ecx; ret`: register addition.
- `sub eax, ecx; ret`: register subtraction.
- `neg eax; ret`: negate.
- `xor eax, eax; ret`: zero a register.

15.3.4 Stack pivots

- `xchg eax, esp; ret`: pivot ESP to attacker-controlled address in EAX.
- `mov esp, ebp; pop ebp; ret`: standard epilogue reused as pivot if EBP is controlled.
- `add esp, offset; ret`: skip stack slots (useful for alignment or jumping over bad characters).
- `pop esp; ret`: direct ESP control from the stack.
- `leave; ret`: equivalent to `mov esp, ebp; pop ebp; ret`.

15.3.5 API call setup

- `push eax; call [ebx];` call through import pointer with argument.
- `pushad; ret`: push all registers, then continue chain. Useful for setting up a complex API call frame.
- `mov edx, [eax]; call [edx+offset];` virtual-call gadget reused for controlled dispatch.

15.4 Fully annotated example

15.4.1 Reading a single gadget

```
; At address 0x7C345678 inside a system DLL:  
pop     eax  
pop     ebx  
ret
```

Annotated:

```
pop eax  
; Load the next stack slot into EAX.
```

```

pop ebx
; Load the following stack slot into EBX.

ret
; Pop the next address and transfer control.

```

Stack layout when this gadget executes:

```

ESP+00: value for EAX
ESP+04: value for EBX
ESP+08: address of next gadget

```

This gadget consumes three stack slots: two data values and one return address.

15.4.2 Reading a short ROP chain

Stack layout (controlled by attacker after overflow):

```

ESP+00: 0x7C345678 ; pop eax; pop ebx; ret
ESP+04: 0x00000040 ; -> EAX = PAGE_EXECUTE_READWRITE
ESP+08: 0x7C349ABC ; -> EBX = (consumed/don't care)
          ; also: address of next gadget
... wait, that's wrong. Let me re-layout:

```

Corrected chain with proper slot accounting:

```

ESP+00: 0x7C345678 ; gadget: pop eax; pop ebx; ret
ESP+04: 0x00000040 ; -> EAX (PAGE_EXECUTE_READWRITE)
ESP+08: 0xDEADBEEF ; -> EBX (unused placeholder)
ESP+0C: 0x7C341234 ; gadget: pop ecx; ret
ESP+10: 0x00401000 ; -> ECX (target address)
ESP+14: 0x7C34ABCD ; gadget: mov [ecx], eax; ret
ESP+18: ... ; next gadget or payload

```

Annotated chain:

```

Step 1: pop eax; pop ebx; ret
        EAX = 0x40 (PAGE_EXECUTE_READWRITE)
        EBX = 0xDEADBEEF (don't care)
        ret -> 0x7C341234

Step 2: pop ecx; ret
        ECX = 0x00401000 (target address for write)
        ret -> 0x7C34ABCD

Step 3: mov [ecx], eax; ret
        Write 0x40 to address 0x00401000
        ret -> next gadget

```

The chain writes a protection constant to a known address. This might be part of setting up arguments for `VirtualProtect` on the stack before calling it.

15.5 Reverse engineering thought process

15.5.1 Identifying gadgets in a binary

Gadgets exist wherever a `ret` instruction appears. Scan backward from each `ret` to find useful instruction sequences:

1. Find all `ret` (`0xC3`) and `ret N` (`0xC2 xx xx`) bytes.
2. Disassemble backward 1–6 bytes from each `ret`.
3. Check whether the preceding bytes form valid, useful instructions.
4. Record the gadget address, instructions, and stack slot consumption.

15.5.2 Mid-instruction gadgets

x86 is variable-length. A `ret` byte (`0xC3`) inside a longer instruction's encoding can be the start of an unintended gadget:

```
Intended instruction at 0x7C345670:
  68 C3 50 90 7C    push 7C9050C3h
```

```
Unintended gadget at 0x7C345671:
  C3                ret
```

```
Unintended gadget at 0x7C345671 with prefix:
  Starting at 0x7C345670:
  ...
```

Better example:

```
Intended instruction at 0x7C345670:
  8B 45 C3          mov eax, [ebp-3Dh]
```

```
Unintended: starting at 0x7C345672:
  C3                ret
```

```
Or:
  B8 5B 5D C3 90    mov eax, 90C35D5Bh
```

```
Unintended at offset +1:
  5B                pop ebx
  5D                pop ebp
  C3                ret
```

The last example is important: the intended instruction `mov eax, 90C35D5Bh` contains the bytes `5B 5D C3` at an offset. Disassembling from that offset yields `pop ebx; pop ebp; ret`, a valid and useful gadget that never appears in the source code or compiler output.

15.5.3 Evaluating gadget quality

Not every gadget is usable:

Good gadget:

- Minimal side effects (only changes target registers)
- No conditional branches
- Predictable stack consumption
- Located in a non-ASLR module or ASLR-bypassed module

Bad gadget:

- Clobbers registers needed by later gadgets
- Contains memory accesses that may fault
- Has conditional branches that change stack consumption
- Located in an ASLR module without an info leak

15.5.4 Chain analysis template

Gadget address:

Instructions:

Stack slots consumed: (pops + 1 for ret, or pops + 1 + N/4 for ret N)

Registers modified:

Registers clobbered (unwanted side effects):

Memory reads:

Memory writes:

Preconditions (registers/memory that must be valid):

15.6 Common mistakes

- Forgetting that ret itself consumes a stack slot (the next gadget address).
- Miscalculating stack consumption when ret N is used. ret 8 pops the return address *and* removes 8 additional bytes from the stack.
- Assuming gadgets only exist at function boundaries. Mid-instruction gadgets are often the most useful.
- Ignoring register clobber. A gadget that sets ECX but also clobbers EAX may break a chain that depends on EAX from a prior gadget.
- Treating pushad as pushing to the ROP chain stack. pushad pushes onto ESP at the time it executes, which may or may not be the chain stack.
- Confusing intended code semantics with gadget semantics. A gadget is used for its side effect, not its original purpose.

15.7 Exercises

Given this stack layout after a buffer overflow:

```

ESP+00: 0x10015442    ; pop eax; ret
ESP+04: 0x00000201    ; -> EAX
ESP+08: 0x10017753    ; pop ecx; ret
ESP+0C: 0x1002A148    ; -> ECX (writable address in .data)
ESP+10: 0x1001AABB    ; mov [ecx], eax; ret
ESP+14: 0x10015442    ; pop eax; ret
ESP+18: 0x7C801D7B    ; -> EAX (address of VirtualProtect)
ESP+1C: 0x10012345    ; push eax; ret

```

Questions:

- What value is written, and where?
- What does the chain do after the write?
- Why is `push eax; ret` useful at the end?
- How many total stack slots does this chain consume?

15.8 Challenge problems

Overflow payload in hex, starting at saved return address:

```

44 55 01 10    ; ESP+00
40 00 00 00    ; ESP+04
53 77 01 10    ; ESP+08
48 A1 02 10    ; ESP+0C
BB AA 01 10    ; ESP+10
42 54 01 10    ; ESP+14
7B 1D 80 7C    ; ESP+18
CC CC CC CC    ; ESP+1C

```

Given the following gadget catalog (all from a non-ASLR module at `0x1001xxxx`):

```

0x10015544: pop eax; ret
0x10015442: pop eax; ret          (duplicate at different address)
0x10017753: pop ecx; ret
0x1001AABB: mov [ecx], eax; ret
0x10015442: pop eax; ret

```

And: `0x7C801D7B` is the address of `kernel32!VirtualProtect`.

Trace the chain step by step. What is the attacker trying to accomplish? What is missing from this chain for a complete DEP bypass?

15.9 Solutions with reasoning

Exercise solution:

Step-by-step trace:

1. `pop eax; ret` at `0x10015442`: `EAX = 0x201`, `ret` to `0x10017753`.
2. `pop ecx; ret` at `0x10017753`: `ECX = 0x1002A148`, `ret` to `0x1001AABB`.
3. `mov [ecx], eax; ret` at `0x1001AABB`: write `0x201` to address `0x1002A148`, `ret` to `0x10015442`.
4. `pop eax; ret` at `0x10015442`: `EAX = 0x7C801D7B` (`VirtualProtect`), `ret` to `0x10012345`.
5. `push eax; ret` at `0x10012345`: push `0x7C801D7B` onto the stack, then `ret` pops it and transfers control to `VirtualProtect`.

The chain writes `0x201` (`PAGE_EXECUTE_READWRITE | MEM_COMMIT` flag bits) to a writable `.data` address, then redirects execution to `VirtualProtect`. The `push eax; ret` pattern is a standard way to call an API whose address is in a register: pushing it makes it the next return address, and `ret` “calls” it.

The chain consumes 8 stack slots (`ESP+00` through `ESP+1C`).

Challenge solution:

The payload bytes are little-endian dword addresses. Reading them:

```
ESP+00: 0x10015544  pop eax; ret
ESP+04: 0x00000040  -> EAX = 0x40 (PAGE_EXECUTE_READWRITE)
ESP+08: 0x10017753  pop ecx; ret
ESP+0C: 0x1002A148  -> ECX = writable .data address
ESP+10: 0x1001AABB  mov [ecx], eax; ret
ESP+14: 0x10015442  pop eax; ret
ESP+18: 0x7C801D7B  -> EAX = VirtualProtect address
ESP+1C: 0xCCCCCCCC  -> padding / int3 marker
```

The chain writes the `PAGE_EXECUTE_READWRITE` constant to a `.data` address, then loads `VirtualProtect`’s address into `EAX`. At `ESP+1C` the chain has `0xCCCCCCCC` (int3 breakpoints), suggesting the chain is incomplete or the attacker is debugging.

For a complete DEP bypass via `VirtualProtect(address, size, newProtect, &oldProtect)`, the chain still needs to:

- Set up the four arguments on the stack (or in the right locations).
- Point the return address of `VirtualProtect` to the shellcode.
- Provide a writable address for the `lpflOldProtect` output parameter.
- Actually transfer control to `VirtualProtect` (the current chain loads it into `EAX` but does not call it yet).

The `0x40` value written to `.data` is likely being staged for later use as the `flNewProtect` argument when the full `VirtualProtect` call frame is assembled.